SMART EXAM LAB MANAGEMENT SYSTEM

An Operating Systems Project Report

Submitted by

Rajesh Kumar Gautam — 24BCE0326

Shourya Thakur — 24BCI0171

Priyanshu Gautam — 24BCE0317

Submitted to

Prof. Geraldine Bessie Amali

School of Computer Science and Engineering (SCOPE)

Contents

Abstract

1. Introduction

2. Problem Statement

3. Existing System / Approach / Method

3.1 Drawbacks / Limitations of the Existing System

4. Proposed / Developed Model

4.1 Design

4.2 Module-wise Description

4.3 Implementation

5. OS Concepts Used

6. Results and Discussion

7. Conclusion, Limitations and Possible Enhancements

8. References

Appendix: Sample Code

Abstract

This project presents the Smart Exam Lab Management System, a client-server application written in C that automates the preparation and supervision of a computer laboratory during examinations. A central server receives periodic health updates from an agent running on every lab PC over TCP/IP, evaluates each machine against configurable thresholds for CPU utilization, RAM usage, network connectivity and required-software availability, and maintains a live status dashboard. Students are allocated to eligible PCs by a deterministic, transaction-protected algorithm that can never assign the same PC twice. Alerts are raised for high resource usage, missing software and client disconnections, and complete exam reports with CSV exports are generated from the SQLite database. The system supports a simulation mode for single-machine demonstration and a real multi-computer mode for Windows 10/11 laboratories.

1. Introduction

Conducting a laboratory examination involves far more than setting the question paper. Before the exam, every computer must be checked: is it switched on and reachable, is it fast enough, is the required software installed, and is it already occupied? During the exam, failures must be noticed quickly, and after the exam a record of who sat where, and what went wrong, is needed. In most laboratories these tasks are performed manually by staff walking between rows of computers.

The Smart Exam Lab Management System replaces that manual process with software. It is implemented in C using Winsock2 for networking, Windows APIs for system metrics, and SQLite for persistent storage, and it directly demonstrates core Operating Systems concepts: process management, memory management, CPU utilization monitoring, resource allocation, file management and networking.

2. Problem Statement

Manual pre-exam laboratory preparation is slow, error-prone and does not scale. PC failures are usually discovered by students after the exam has started; there is no systematic record of machine health, seat allocation or incidents; and assigning students to working computers on the spot wastes examination time. The problem addressed is: how can a laboratory automatically verify the health of every PC, allocate students only to machines that are genuinely ready, monitor the lab in real time, and produce a trustworthy post-exam report — using Operating Systems mechanisms rather than manual checking?

3. Existing System / Approach / Method

In the existing approach, laboratory staff prepare for an exam manually. Each PC is visited individually, switched on, and checked by opening the required applications. A handwritten

or spreadsheet list maps students to PC numbers. During the exam, problems are reported verbally by students and handled ad hoc. Attendance and incident records, if kept at all, are maintained on paper.

3.1 Drawbacks / Limitations of the Existing System

- Time-consuming: checking dozens of PCs one by one can take hours.
- Unreliable: a PC that worked yesterday may fail today; manual checks give no live status.
- No early warning: high CPU/RAM usage or missing software is found during the exam, not before.
- Allocation conflicts: two students can be sent to the same PC, or to a PC that is down.
- No audit trail: after the exam there is no dependable record of assignments or incidents.
- Not scalable: the effort grows linearly with the number of computers and students.

4. Proposed / Developed Model

The proposed system has three components. (1) A central server runs on the administrator's computer: it listens for client connections, stores all state in SQLite, evaluates health, performs allocation, manages alerts and generates reports, and provides a terminal administrator console. (2) A lightweight client agent runs on every lab PC: it measures its own CPU utilization (interval-based, using GetSystemTimes), RAM usage (GlobalMemoryStatusEx), running-process count (ToolHelp32), network state and required-software presence, and reports them to the server every few seconds over TCP. (3) A simulation mode runs virtual agents on one computer so the complete workflow can be demonstrated without a physical laboratory; simulated metrics are always labelled as simulation.

4.1 Design

Communication uses a documented, versioned protocol: each message is framed with a 4-byte big-endian length prefix followed by a field-based ASCII payload (version | type | pc_id | timestamp | key=value pairs). Raw C structures are never transmitted, avoiding padding and endianness problems. The server uses one thread per client connection plus a sweeper thread that detects heartbeat timeouts, so one slow client never blocks the others. Health status follows strict precedence: DISCONNECTED (stale heartbeat) takes priority over MAINTENANCE, then NOT_READY (network down or software missing), then ASSIGNED, then WARNING (CPU or RAM above the configured threshold), and finally READY.

The database is normalized into computers, students, exams, assignments, health_checks, alerts, config and audit tables, with foreign keys enforced and partial unique indexes guaranteeing that an active assignment is unique per PC and per student-exam pair.

4.2 Module-wise Description

Module	Responsibility
config.c	Loads key=value configuration: port, thresholds, token, intervals.
protocol.c	Length-prefixed framing, safe parsing and validation of messages.
database.c	SQLite schema and all prepared-statement data access.
monitor.c	Windows/Linux metrics collection, simulation backend, health rules.
allocation.c	Deterministic student-to-PC allocation inside a transaction.
alerts.c	Deduplicated alert raise / acknowledge / resolve lifecycle.
reports.c	Exam reports and CSV exports generated from stored data.
server.c	Thread-per-client TCP server, heartbeat sweeper, simulation launcher.
client.c	Agent loop: register, heartbeat, reconnect with backoff.
main_server.c	Terminal administrator console (16 operations).
main_client.c	Agent entry point and command-line options.

4.3 Implementation

The system is implemented in standard C (C11) across roughly 1,800 lines of project code, plus the bundled SQLite amalgamation. The server and client are separate executables (exam_server.exe, exam_client.exe) built with MinGW GCC, linking ws2_32 for Winsock. Clients authenticate with a per-lab provisioning token carried in every message; all inputs are validated (message size limits, PC-ID character set, percentage ranges) and all SQL uses bound parameters. Allocation runs inside a BEGIN IMMEDIATE transaction: candidates are fresh, READY, unassigned PCs; the winner is the lowest CPU+RAM score with pc_id as deterministic tie-break, giving O(P log P) behaviour for P PCs and making double-assignment impossible even under concurrent requests.

5. OS Concepts Used

Concept	How the project demonstrates it
Process management	Agent counts running processes (ToolHelp32 snapshot / /proc); server runs one thread per client. The project observes processes; it does not implement the OS scheduler.
Memory management	RAM utilization and available memory read through GlobalMemoryStatusEx (Windows) — real memory statistics from the OS.
CPU management	CPU utilization computed from

	GetSystemTimes deltas over an interval; thresholds classify PCs. Monitoring utilization is distinct from CPU scheduling, which remains the kernel's job.
Resource management	PCs are managed resources whose state (ready / assigned / maintenance) is tracked centrally.
File management	SQLite database, configuration files, report and CSV files persist all state across restarts.
Networking	TCP/IP client-server communication with framing, heartbeats, timeouts and reconnection. A successful ping alone would not prove exam services work — the real liveness check is the agent's TCP session.
Scheduling and allocation	A documented allocation policy reserves eligible PCs for students — resource allocation, distinguished from process scheduling.

6. Results and Discussion

The system was built and verified on the development machine using the same C sources that produce the Windows executables. All four automated test suites pass: health evaluation (9 cases covering every status and the precedence rules), protocol parsing and malformed-input rejection, database creation and persistence across restarts, and the allocation matrix (exam not active, unknown student, duplicate allocation, high-CPU / high-RAM / missing-software / stale-heartbeat exclusion, no-PC-available, and release with reassignment).

In an end-to-end integration run, the server with five simulated agents registered all five PCs with correct live statuses (including one WARNING PC at 93% CPU, which raised a HIGH_CPU alert). Starting the demo exam and allocating two students followed the documented policy exactly — the lowest-load READY PCs were chosen — and the generated exam report and CSV exports matched the database contents. A separate run of the real (non-simulated) client registered with genuine OS metrics (hostname, CPU and RAM read from the operating system). After a server restart, all records and active assignments were still present, confirming persistence.

Scope note, stated plainly: at the time of writing, the Windows executables have been compiled (MinGW cross-compiler; native build script provided) and the full workflow has been exercised in simulation mode and with a real-metrics client on the development machine. A run across multiple physical Windows PCs follows the documented deployment procedure but is not claimed here as completed test evidence.

7. Conclusion, Limitations and Possible Enhancements

The Smart Exam Lab Management System automates laboratory health checks, student-to-PC allocation, real-time monitoring, alerting and reporting in a single C application, and demonstrates the targeted Operating Systems concepts through working code rather than description alone. Its allocation is deterministic and race-safe, its data survives restarts, and its workflow can be demonstrated on one machine or deployed across a real lab LAN.

Limitations: (i) transport is plain TCP without encryption — acceptable for a trusted lab LAN prototype, but production use needs TLS; (ii) the server is a single point of failure; (iii) client identity rests on a shared provisioning token rather than per-PC certificates; (iv) the administrator interface is terminal-based.

Possible enhancements: TLS with per-PC certificates, a web dashboard over the same database, automatic reassignment with admin approval when a PC fails mid-exam, Wake-on-LAN power-on of lab PCs, and historical health analytics to predict failures before exam day.

8. References

- Silberschatz, Galvin & Gagne — Operating System Concepts, Wiley.
- Microsoft Learn — Winsock2, GetSystemTimes, GlobalMemoryStatusEx, ToolHelp32 API documentation.
- SQLite Documentation — C API, WAL mode, partial indexes (sqlite.org).
- Beej's Guide to Network Programming — TCP stream handling and framing.
- MinGW-w64 project documentation — GCC toolchain for Windows.

Appendix: Sample Code

A. Allocation core (src/allocation.c) — candidate selection runs inside the BEGIN IMMEDIATE transaction described in section 4.3:

```
sqlite3_prepare_v2(g_db,

  "SELECT
c.pc_id,c.cpu_pct,c.ram_pct,c.net_ok,c.software_ok,c.last_seen,c.maintenance "

  "FROM computers c WHERE c.maintenance=0 AND NOT EXISTS "

  "(SELECT 1 FROM assignments a WHERE a.pc_id=c.pc_id AND a.released_at IS NULL)",
-1, &q, NULL);

while (sqlite3_step(q) == SQLITE_ROW) {

    /* skip stale records, evaluate health, keep READY PCs only */

    PcStatus st = health_evaluate(&m, th, 0, maint, age);

    if (st != ST_READY) continue;

    cands[nc].score = m.cpu_pct + m.ram_pct;   /* lowest load wins */

}

qsort(cands, nc, sizeof(Cand), cmp_cand);      /* tie-break: pc_id */

/* INSERT assignment; partial unique index ux_active_pc is the hard guarantee */
```

B. Health evaluation (src/monitor.c) — the precedence rules in one function:

```
PcStatus health_evaluate(const HealthMetrics *m, const HealthThresholds *th,

                         int assigned, int maintenance, long age_s) {

    if (age_s > th->heartbeat_timeout_s) return ST_DISCONNECTED;

    if (maintenance) return ST_MAINTENANCE;

    if (!m->net_ok) return ST_NOT_READY;

    if (th->require_software && !m->software_ok) return ST_NOT_READY;

    if (assigned) return ST_ASSIGNED;

    if (m->cpu_pct >= th->cpu_warn_pct || m->ram_pct >= th->ram_warn_pct)

        return ST_WARNING;

    return ST_READY;

}
```

C. Protocol framing (src/protocol.c): a 4-byte big-endian length prefix is sent before every payload; send/recv loop until all bytes have moved, so partial TCP reads and writes are handled correctly.